



Escuela de Ingeniería en Computación
Ingeniería en Computación
IC6821 - Diseño de Software

Proyecto: Análisis de Requerimientos

Plataforma para la Enseñanza de Programación

Mishelle Orellana Jiménez
m.orellana.1@estudiantec.cr
2024099469

Amanda Montero Zúñiga
amontzun24@gmail.com
2021579522

San José, Costa Rica
Marzo 2026

Índice general

1. Introducción	3
1.1. Introducción	3
1.2. Antecedentes y contexto	3
1.3. Descripción del problema	4
1.4. Descripción de la solución propuesta	4
1.5. Justificación de la solución	5
1.6. Identificación de Actores	5
1.6.1. Estudiante	5
1.6.2. Profesor	6
1.6.3. Repositorio Git	6
1.6.4. Módulo de validación criptográfica	7
1.6.5. Motor de casos de prueba	7
1.7. Identificación de Escenarios	8
1.8. Requerimientos Funcionales de Alto Nivel	14
1.9. Análisis de Casos de Uso	19
1.9.1. CU-01: Iniciar sesión en la plataforma	19
1.9.2. CU-02: Consultar dashboard y asignaciones pendientes	19
1.9.3. CU-03: Crear o abrir archivo Python en la IDE	20
1.9.4. CU-04: Editar archivo Python dentro de la IDE	20
1.9.5. CU-05: Bloquear edición de archivo modificado externamente	21
1.9.6. CU-06: Ejecutar casos de prueba	22
1.9.7. CU-07: Realizar commit de cambios	22
1.9.8. CU-08: Gestionar proyectos colaborativos	23
1.9.9. CU-09: Crear asignación	23
1.9.10. CU-10: Configurar casos de prueba	24
1.9.11. CU-11: Agregar comentarios al trabajo del estudiante	25
1.9.12. CU-12: Gestionar llaves de validación del curso	25
1.10. Requerimientos No Funcionales	26
1.10.1. Funcionalidad (Functionality)	26
1.10.2. Usabilidad (Usability)	27
1.10.3. Confiabilidad (Reliability)	29
1.10.4. Rendimiento (Performance)	30
1.10.5. Soporte (Supportability)	31

1.10.6. Restricciones adicionales (+)	33
1.11. Diagrama de Casos de Uso	34
Referencias	36

Capítulo 1

Introducción

1.1. Introducción

El aprendizaje de la programación en los primeros cursos universitarios constituye una etapa fundamental en la formación de estudiantes de Computación, ya que en esta fase se desarrollan las bases del pensamiento lógico, la resolución de problemas y la comprensión estructurada de algoritmos. Asignaturas como *Introducción a la Programación* y *Taller de Programación* tienen como propósito que el estudiante construya dichas habilidades mediante la práctica constante, el análisis de errores y la elaboración progresiva de soluciones propias. Sin embargo, en los últimos años han surgido nuevos retos asociados al uso creciente de herramientas externas que facilitan la obtención inmediata de código, lo que puede debilitar el proceso de aprendizaje cuando se utilizan como sustituto del razonamiento individual.

En este contexto, se plantea el desarrollo de una plataforma académica orientada específicamente a cursos iniciales de programación en Python. La propuesta contempla un entorno controlado compuesto por un *dashboard* de cursos y asignaciones, una IDE especializada, mecanismos de validación criptográfica para garantizar la integridad de los archivos fuente, integración con Git para control de versiones y soporte para casos de prueba y trabajo colaborativo. La intención principal es brindar un espacio donde el estudiante pueda practicar programación de manera auténtica, reduciendo la dependencia de código externo y fortaleciendo la integridad académica dentro del proceso formativo.

1.2. Antecedentes y contexto

En los cursos de primer ingreso de la carrera de Computación, el aprendizaje de la programación se enfrenta actualmente a un escenario distinto al de años anteriores, debido a la disponibilidad masiva de herramientas digitales capaces de generar código automáticamente. Aunque estas tecnologías pueden resultar útiles en etapas avanzadas o en contextos profesionales, su uso en cursos intro-

ductorios puede afectar directamente la construcción de conocimientos básicos, ya que muchos estudiantes pueden llegar a resolver ejercicios sin comprender la lógica que hay detrás de las soluciones. Esto resulta especialmente problemático en cursos cuyo objetivo principal es formar habilidades esenciales, más que únicamente obtener un resultado funcional.

Dentro de este contexto académico, surge la necesidad de diseñar una herramienta que responda específicamente a las necesidades de asignaturas iniciales de programación. El proyecto se orienta a estudiantes de primer año que cursan materias introductorias basadas en Python, lenguaje ampliamente utilizado por su claridad sintáctica y su valor pedagógico. Además, el sistema debe ajustarse a dinámicas reales del proceso educativo, permitiendo la gestión de asignaciones por curso, la revisión docente, la ejecución de pruebas y el trabajo colaborativo, sin incorporar elementos fuera del alcance definido, como mensajería interna, publicación de notas o mecanismos tradicionales de entrega separados del entorno de desarrollo

1.3. Descripción del problema

El problema principal que motiva este proyecto es la dificultad para garantizar que los estudiantes de cursos introductorios realmente aprendan a programar mediante la construcción de sus propias soluciones. En la práctica, el acceso a herramientas externas capaces de generar código ha facilitado que parte del trabajo académico pueda resolverse sin un proceso genuino de razonamiento, análisis o experimentación por parte del estudiante. Como consecuencia, se debilita el desarrollo de competencias fundamentales como la descomposición de problemas, la escritura correcta de código, la depuración y la comprensión de estructuras básicas de programación.

Adicionalmente, los entornos tradicionales no siempre ofrecen mecanismos para verificar que los archivos fuente hayan sido creados y modificados únicamente dentro de un espacio autorizado. Esto permite que el código sea editado externamente, reutilizado sin control o incorporado desde otras fuentes sin trazabilidad suficiente. A esto se suma la ausencia de restricciones concretas sobre acciones como copiar y pegar dentro del entorno de programación, así como la necesidad de que el docente disponga de una herramienta que le permita asignar ejercicios, revisar trabajo, agregar comentarios y fomentar colaboración, todo ello dentro de un sistema enfocado específicamente en el aprendizaje auténtico de la programación.

1.4. Descripción de la solución propuesta

Como respuesta a la problemática planteada, se propone el desarrollo de una plataforma académica para la enseñanza de programación en Python, diseñada para funcionar como un entorno controlado de trabajo para estudiantes de primer ingreso. La solución incluye un *dashboard* desde el cual el estudiante podrá

visualizar sus cursos y asignaciones pendientes, así como acceder a una IDE especializada para la creación y edición de archivos `.py`. Dicha IDE restringirá acciones que faciliten la incorporación de código externo, particularmente mediante la deshabilitación de operaciones de copiado y pegado, con el objetivo de promover la construcción directa de soluciones dentro del sistema.

Asimismo, la plataforma incorporará un mecanismo de validación criptográfica basado en hashes o firmas asociadas a los archivos fuente, de forma que cada archivo `.py` mantenga evidencia de haber sido modificado únicamente desde la IDE autorizada. Si un archivo es alterado fuera del sistema, la plataforma detectará la inconsistencia y bloqueará su edición o uso hasta que se restablezca su integridad. El sistema también integrará control de versiones mediante Git, permitirá la ejecución de casos de prueba definidos por el profesor, soportará proyectos colaborativos en tiempo real o por versiones, y ofrecerá al docente herramientas para asignar ejercicios, administrar parámetros de validación y agregar comentarios sobre el trabajo realizado por los estudiantes.

1.5. Justificación de la solución

La solución propuesta se justifica por la necesidad de proteger el proceso formativo en los cursos iniciales de programación, asegurando que el estudiante desarrolle habilidades reales y no dependa de herramientas externas para resolver sus ejercicios. En una etapa temprana de aprendizaje, resulta fundamental que la práctica se realice dentro de condiciones controladas que favorezcan la comprensión progresiva de conceptos, la corrección de errores y la elaboración autónoma de soluciones. Un entorno diseñado específicamente para este propósito puede contribuir significativamente a fortalecer tanto la integridad académica como la calidad del aprendizaje.

Además, la propuesta no se limita a restringir conductas indeseadas, sino que también incorpora funcionalidades valiosas para el proceso educativo, como el uso de casos de prueba, la retroalimentación docente, el control de versiones y la colaboración en proyectos grupales. Esto convierte a la plataforma en una herramienta integral para apoyar la enseñanza de la programación en Python desde una perspectiva pedagógica y técnica. En consecuencia, el proyecto resulta pertinente porque responde a una necesidad concreta del contexto universitario actual, alinea la tecnología con los objetivos de formación y ofrece una solución especializada para mejorar la experiencia de aprendizaje en cursos introductorios de Computación.

1.6. Identificación de Actores

1.6.1. Estudiante

El estudiante es el actor principal de la plataforma, ya que utiliza el sistema para acceder a sus cursos, revisar asignaciones pendientes, desarrollar soluciones en la IDE de Python y trabajar en proyectos individuales o colaborativos. Su

interacción con el sistema se limita al entorno controlado de la plataforma, de manera que pueda programar, ejecutar pruebas y gestionar versiones mediante Git, sin modificar archivos fuera del sistema ni introducir código externo mediante acciones de copiado y pegado. Este actor representa al usuario que aprende programación y que debe desarrollar sus ejercicios dentro de condiciones que favorezcan el aprendizaje genuino.

- **Tipo:** Humano

- **Permisos:** El estudiante puede iniciar sesión en la plataforma, visualizar sus cursos y asignaciones pendientes, abrir la IDE de Python, crear y editar archivos `.py` únicamente dentro del entorno del sistema, ejecutar casos de prueba, realizar *commits* a un repositorio Git asociado al curso o proyecto, participar en proyectos colaborativos cuando haya sido autorizado y visualizar los comentarios que el profesor haya realizado sobre su trabajo. No puede copiar ni pegar código dentro de la IDE, no puede modificar archivos Python fuera de la plataforma, no puede cambiar llaves criptográficas ni mecanismos de verificación, no puede asignar ejercicios ni gestionar usuarios.

1.6.2. Profesor

El profesor es el actor encargado de la administración académica del sistema. Su función consiste en crear y gestionar cursos, asignar ejercicios o proyectos, configurar casos de prueba, administrar las llaves criptográficas o mecanismos de validación que protegen la integridad de los archivos `.py`, revisar el trabajo de los estudiantes y proporcionar retroalimentación mediante comentarios. También puede habilitar proyectos colaborativos y monitorear el historial de cambios mediante integración con Git. Este actor tiene un rol de supervisión y control sobre el entorno de aprendizaje.

- **Tipo:** Humano

- **Permisos:** El profesor puede crear cursos, publicar asignaciones, configurar casos de prueba, revisar soluciones desarrolladas por los estudiantes dentro del sistema, agregar comentarios de retroalimentación, habilitar proyectos grupales, asociar o gestionar las llaves públicas necesarias para la validación de archivos, visualizar historial de *commits* y administrar los parámetros académicos del entorno. No utiliza la plataforma como canal de mensajería ni como sistema de publicación de notas finales, ya que dichas funciones no forman parte del alcance del proyecto.

1.6.3. Repositorio Git

El repositorio Git actúa como un sistema externo de control de versiones que permite registrar cambios realizados por los estudiantes y equipos de trabajo. Su propósito es brindar trazabilidad sobre la evolución de los archivos fuente,

facilitar la realización de *commits* y apoyar el trabajo colaborativo en proyectos grupales. Aunque no toma decisiones académicas, es un componente clave para mantener el historial de modificaciones y respaldar la integridad del desarrollo.

- **Tipo:** Sistema
- **Permisos:** El repositorio Git recibe *commits* generados desde la plataforma, almacena versiones de los archivos permitidos, mantiene historial de cambios, permite la sincronización del trabajo colaborativo y provee información de versionado al sistema. No crea cursos, no valida aprendizaje, no asigna tareas y no modifica archivos por iniciativa propia.

1.6.4. Módulo de validación criptográfica

Este actor representa el subsistema encargado de verificar la integridad de los archivos `.py` mediante firmas, hashes o mecanismos criptográficos controlados por el profesor. Su función es asegurar que los archivos hayan sido creados o modificados únicamente desde la IDE autorizada, y detectar cualquier alteración externa. Cuando un archivo no cumple con la verificación correspondiente, el sistema debe impedir su edición o ejecución hasta que se restablezca su integridad a través del entorno legítimo.

- **Tipo:** Sistema
- **Permisos:** El módulo puede generar y validar hashes o firmas de archivos `.py`, insertar o verificar el comentario de identificación en la primera línea del archivo, detectar modificaciones externas, bloquear la edición de archivos alterados fuera del entorno permitido y actualizar la firma cuando un cambio se realiza legítimamente desde la IDE. No puede asignar tareas, crear cursos, comentar ejercicios ni modificar permisos académicos.

1.6.5. Motor de casos de prueba

El motor de casos de prueba es el sistema encargado de ejecutar automáticamente pruebas definidas por el profesor sobre el código del estudiante. Su propósito es permitir la validación funcional del programa dentro del entorno controlado, ayudando al estudiante a comprobar si su solución cumple con los requisitos establecidos en el ejercicio. Este actor contribuye al aprendizaje práctico, ya que proporciona una verificación objetiva del comportamiento del código.

- **Tipo:** Sistema
- **Permisos:** El motor puede ejecutar casos de prueba sobre programas Python, mostrar resultados al estudiante, reportar éxitos o errores de ejecución y utilizar configuraciones establecidas por el profesor. No puede editar código, asignar cursos, administrar usuarios ni modificar archivos fuera del proceso de prueba.

1.7. Identificación de Escenarios

Los escenarios descritos a continuación documentan instancias concretas de interacción entre actores y el sistema para flujos representativos de trabajo. Cada tabla sigue el formato de la Figura 4-6 del libro de Bruegge y Dutoit (*Object-Oriented Software Engineering*, p. 133), notación canónica de escenarios en el proceso RUP.

Escenario 1: entregaTareaIndividual

<i>Nombre del escenario</i>	entregaTareaIndividual
<i>Actores participantes</i>	juan:Estudiante
<i>Flujo de eventos</i>	1. Juan inicia sesión en la plataforma y accede a su <i>dashboard</i>. 2. Selecciona la asignación “<i>Introducción a listas</i>” del curso de Programación I. 3. La IDE se abre y Juan comienza a escribir su solución directamente en el editor. El sistema bloquea automáticamente el pegado de texto externo. 4. Juan ejecuta los casos de prueba configurados por el profesor; el motor reporta dos pruebas exitosas y una fallida. 5. Juan corrige el error en el código y vuelve a ejecutar las pruebas, que ahora pasan en su totalidad. 6. Juan selecciona la opción <i>Entregar</i>. El sistema calcula la firma digital del archivo <code>.py</code> y la registra como entrega oficial. 7. Juan recibe confirmación de que la tarea fue entregada correctamente.

Figura: Escenario *entregaTareaIndividual* para el caso de uso *Editar y entregar tarea*.

Escenario 2: profesorPublicaAsignacion

<i>Nombre del escenario</i>	profesorPublicaAsignacion
<i>Actores participantes</i>	maria:Profesor, motorPruebas:MotorDeCasosDePrueba
<i>Flujo de eventos</i>	1. María inicia sesión como profesora y accede al curso <i>Programación I</i>. 2. Selecciona <i>Nueva asignación</i> e ingresa el título, descripción, fecha y hora límite de entrega. 3. María activa un plazo de prueba de 48 horas previo al cierre para que los estudiantes verifiquen su solución sin poder entregar. 4. Define tres casos de prueba (entradas y salidas esperadas) que el motor almacena y asocia a la asignación. 5. María confirma la publicación; el sistema hace visible la asignación en el <i>dashboard</i> de los estudiantes. 6. Los estudiantes reciben una notificación indicando que existe una nueva tarea disponible.

Figura: Escenario profesorPublicaAsignacion para el caso de uso Crear asignación.

Escenario 3: `deteccionArchivoAlterado`

<i>Nombre del escenario</i>	<code>deteccionArchivoAlterado</code>
<i>Actores participantes</i>	<code>carlos:Estudiante,</code> <code>modVal:ModuloValidacionCriptografica</code>
<i>Flujo de eventos</i>	1. Carlos intenta abrir en la IDE el archivo <code>solucion.py</code> de una asignación activa. 2. El módulo de validación criptográfica lee la firma del encabezado del archivo y recalcula el hash del contenido actual. 3. El sistema detecta una discrepancia: el archivo fue editado con un editor externo. 4. La IDE muestra un mensaje de advertencia y bloquea la edición del archivo. 5. Carlos no puede continuar trabajando en ese archivo hasta que el profesor restablezca su integridad o se genere un archivo limpio para la asignación.

Figura: Escenario `deteccionArchivoAlterado` para el caso de uso Bloquear edición de archivo modificado externamente.

Escenario 4: commitProyectoGrupal

<i>Nombre del escenario</i>	commitProyectoGrupal
<i>Actores participantes</i>	ana:Estudiante, repoGit:RepositorioGit
<i>Flujo de eventos</i>	1. Ana accede al proyecto colaborativo “<i>Gestión de Inventario</i>” habilitado por su profesora. 2. La plataforma carga el entorno compartido y Ana visualiza los archivos del proyecto en el explorador de la IDE. 3. Ana edita <code>inventario.py</code> añadiendo la función de búsqueda binaria acordada con el equipo. 4. Desde la terminal integrada, ejecuta <code>git pull</code> para sincronizar los cambios de sus compañeros; no se detectan conflictos. 5. Ana ejecuta <code>git commit -m "feat: búsqueda binaria en inventario"</code>. El sistema valida la integridad criptográfica antes de enviar el <i>commit</i>. 6. El repositorio registra la nueva versión y Ana recibe confirmación del <i>commit</i> exitoso.

Figura: Escenario commitProyectoGrupal para el caso de uso Gestionar proyectos colaborativos.

Escenario 5: entregaModificadaConCopiaOriginal

<i>Nombre del escenario</i>	entregaModificadaConCopiaOriginal
<i>Actores participantes</i>	pedro:Estudiante, modVal:ModuloValidacionCriptografica
<i>Flujo de eventos</i>	1. Pedro entrega la asignación “<i>Ordenamiento burbuja</i>” antes del plazo. El sistema genera la firma digital y registra la entrega con marca de tiempo. 2. Pedro detecta un error y decide modificar su solución antes del vencimiento. 3. Pedro edita el archivo en la IDE; el módulo de validación marca el cambio como versión posterior a la entrega. 4. El sistema almacena automáticamente una copia inmutable de la entrega original junto a la nueva versión modificada. 5. Al vencer el plazo, el profesor puede revisar ambas versiones claramente diferenciadas en la plataforma.

Figura: Escenario entregaModificadaConCopiaOriginal para el caso de uso Pre-servación de entrega original.

Escenario 6: plazoPruebaActivado

<i>Nombre del escenario</i>	plazoPruebaActivado
<i>Actores participantes</i>	laura:Profesor, sofia:Estudiante
<i>Flujo de eventos</i>	1. Laura configura una asignación con fecha límite el viernes a las 11:59 p.m. y define un plazo de prueba que inicia el miércoles. 2. El miércoles, el sistema activa automáticamente el plazo de prueba para la asignación. 3. Sofía abre la asignación durante el plazo de prueba; puede ejecutar casos de prueba pero el botón de entrega aparece deshabilitado. 4. Sofía trabaja en su solución, la ejecuta contra los casos de prueba y obtiene retroalimentación del motor. 5. El viernes, al iniciar el período oficial de entrega, el botón se habilita y Sofía realiza su envío formal. 6. El sistema registra la entrega con la firma digital y su marca de tiempo correspondiente.

Figura: Escenario `plazoPruebaActivado` para el caso de uso Plazos de entrega y prueba.

1.8. Requerimientos Funcionales de Alto Nivel

Los siguientes requerimientos funcionales fueron identificados a partir del análisis de actores, escenarios y casos de uso. Cada uno se presenta en una tabla individual con su código de seguimiento, nombre, tipo, descripción, prioridad y dependencias funcionales.

Código de requisito	RF-01
Nombre de requisito	Autenticación por rol
Tipo	Funcional
Descripción	El sistema permite a estudiantes y profesores iniciar sesión con credenciales propias. Tras la validación, redirige al usuario al <i>dashboard</i> según el rol identificado.
Prioridad del requisito	Alta
Dependencias funcionales	—

Código de requisito	RF-02
Nombre de requisito	Dashboard de cursos y asignaciones
Tipo	Funcional
Descripción	El sistema muestra al estudiante los cursos en que está matriculado y las asignaciones pendientes de cada uno. Al profesor le muestra los cursos que administra y el estado general de entregas.
Prioridad del requisito	Alta
Dependencias funcionales	RF-01

Código de requisito	RF-03
Nombre de requisito	Gestión de archivos en la IDE
Tipo	Funcional
Descripción	El sistema permite crear, abrir, guardar y cargar archivos <code>.py</code> exclusivamente dentro del entorno de la IDE. Cualquier archivo de otro tipo o proveniente de fuente externa es rechazado.
Prioridad del requisito	Alta
Dependencias funcionales	RF-01

Código de requisito	RF-04
Nombre de requisito	Restricción de edición controlada
Tipo	Funcional
Descripción	La IDE deshabilita las operaciones de copiar y pegar texto proveniente de fuentes externas. Toda escritura debe realizarse directamente en el editor de la plataforma.
Prioridad del requisito	Alta
Dependencias funcionales	RF-03

Código de requisito	RF-05
Nombre de requisito	Validación criptográfica de archivos
Tipo	Funcional
Descripción	Al crear o modificar un archivo .py, el sistema calcula una firma digital (hash) que se almacena como comentario en el encabezado del archivo. Si el archivo es alterado externamente, la firma no coincide y el sistema bloquea su uso.
Prioridad del requisito	Alta
Dependencias funcionales	RF-03, RF-04

Código de requisito	RF-06
Nombre de requisito	Ejecución de casos de prueba
Tipo	Funcional
Descripción	El estudiante puede ejecutar los casos de prueba configurados por el profesor sobre su código. El motor de pruebas devuelve el resultado de cada caso (éxito o fallo) con la salida obtenida versus la esperada.
Prioridad del requisito	Media
Dependencias funcionales	RF-04, RF-05

Código de requisito	RF-07
Nombre de requisito	Control de versiones Git desde la terminal
Tipo	Funcional
Descripción	La plataforma incluye una terminal interactiva integrada desde la que el estudiante puede ejecutar comandos Git (<code>pull</code> , <code>commit</code> , etc.). El sistema valida la integridad criptográfica de los archivos antes de permitir un <i>commit</i> .
Prioridad del requisito	Alta
Dependencias funcionales	RF-03, RF-05

Código de requisito	RF-08
Nombre de requisito	Proyectos colaborativos
Tipo	Funcional
Descripción	El profesor puede habilitar proyectos grupales a los que se asocian varios estudiantes. Todos los integrantes acceden al mismo espacio de trabajo; los cambios se coordinan mediante el repositorio Git asociado al curso.
Prioridad del requisito	Media
Dependencias funcionales	RF-07, RF-09

Código de requisito	RF-09
Nombre de requisito	Creación y publicación de asignaciones
Tipo	Funcional
Descripción	El profesor puede crear asignaciones indicando título, descripción, fecha y hora de entrega, y activar o no un plazo de prueba previo. Las asignaciones publicadas aparecen en el <i>dashboard</i> de los estudiantes del curso. No se registra ni publica la nota obtenida.
Prioridad del requisito	Alta
Dependencias funcionales	RF-01

Código de requisito	RF-10
Nombre de requisito	Configuración de casos de prueba
Tipo	Funcional
Descripción	El profesor puede asociar casos de prueba a una asignación definiendo entradas, salidas esperadas y condiciones de validación. El motor de casos de prueba utiliza esta configuración para las ejecuciones del estudiante.
Prioridad del requisito	Media
Dependencias funcionales	RF-09

Código de requisito	RF-11
Nombre de requisito	Retroalimentación docente
Tipo	Funcional
Descripción	El profesor puede agregar comentarios sobre el código o el historial de cambios de un estudiante. Los comentarios quedan visibles para el estudiante en la plataforma.
Prioridad del requisito	Media
Dependencias funcionales	RF-09

Código de requisito	RF-12
Nombre de requisito	Gestión de llaves de validación del curso
Tipo	Funcional
Descripción	El profesor puede registrar y actualizar la llave pública o parámetros criptográficos asociados a su curso. El módulo de validación utiliza esta configuración para verificar la integridad de todos los archivos del curso.
Prioridad del requisito	Alta
Dependencias funcionales	RF-05, RF-09

Código de requisito	RF-13
Nombre de requisito	Preservación de la entrega original
Tipo	Funcional
Descripción	Cuando un estudiante entrega una asignación y luego la modifica antes del vencimiento del plazo, el sistema conserva automáticamente una copia inmutable de la primera entrega. El profesor puede acceder a ambas versiones: la entrega original y la última modificación.
Prioridad del requisito	Alta
Dependencias funcionales	RF-03, RF-05

Código de requisito	RF-14
Nombre de requisito	Plazos de prueba configurables
Tipo	Funcional
Descripción	El profesor puede definir un período de prueba previo a la fecha oficial de entrega, durante el cual los estudiantes pueden trabajar en la asignación y ejecutar casos de prueba, pero no realizar la entrega formal. La entrega solo se habilita al inicio del período oficial.
Prioridad del requisito	Alta
Dependencias funcionales	RF-09

Código de requisito	RF-15
Nombre de requisito	Terminal interactiva integrada
Tipo	Funcional
Descripción	La plataforma dispone de una terminal integrada que permite al estudiante ejecutar comandos Git (<code>pull</code> , <code>commit</code>) y otros comandos de consola de Python. La terminal opera dentro del entorno controlado del sistema y no otorga acceso a directorios fuera del proyecto activo.
Prioridad del requisito	Alta
Dependencias funcionales	RF-07

1.9. Análisis de Casos de Uso

1.9.1. CU-01: Iniciar sesión en la plataforma

Objetivo: Permitir que un usuario autorizado acceda al sistema según su rol.

Actor principal: Estudiante / Profesor

Precondiciones:

- El usuario debe estar registrado en la plataforma.

Postcondiciones:

- El usuario accede al *dashboard* correspondiente a su rol.

Flujo principal:

1. El usuario ingresa sus credenciales.
2. El sistema valida la información ingresada.
3. El sistema identifica el rol del usuario.
4. El sistema muestra el *dashboard* correspondiente.

Flujos alternos:

- Si las credenciales son inválidas, el sistema muestra un mensaje de error.
- Si la cuenta no tiene cursos activos, el sistema permite el ingreso pero muestra un panel vacío.

1.9.2. CU-02: Consultar dashboard y asignaciones pendientes

Objetivo: Permitir al estudiante visualizar sus cursos y ejercicios pendientes.

Actor principal: Estudiante

Precondiciones:

- El estudiante ha iniciado sesión.

Postcondiciones:

- El estudiante visualiza la lista de cursos y asignaciones disponibles.

Flujo principal:

1. El estudiante accede al *dashboard*.
2. El sistema consulta los cursos en los que está matriculado.
3. El sistema muestra las asignaciones pendientes asociadas a cada curso.

4. El estudiante selecciona una asignación para trabajar en ella.

Flujos alternos:

- Si no hay asignaciones pendientes, el sistema muestra un mensaje indicándolo.

1.9.3. CU-03: Crear o abrir archivo Python en la IDE

Objetivo: Permitir al estudiante trabajar únicamente con archivos `.py` dentro del entorno controlado.

Actor principal: Estudiante

Precondiciones:

- El estudiante ha ingresado a una asignación válida.

Postcondiciones:

- El archivo `.py` queda abierto y listo para edición dentro de la IDE.

Flujo principal:

1. El estudiante selecciona una asignación.
2. El sistema abre la IDE de Python.
3. El estudiante crea un archivo nuevo o abre uno ya existente.
4. El sistema verifica que el archivo corresponda al formato `.py`.
5. El sistema habilita la edición únicamente dentro de la IDE.

Flujos alternos:

- Si el archivo no es `.py`, el sistema rechaza su apertura.
- Si el archivo fue alterado externamente, el sistema bloquea su edición.

1.9.4. CU-04: Editar archivo Python dentro de la IDE

Objetivo: Permitir la modificación de archivos Python únicamente desde el entorno autorizado.

Actor principal: Estudiante

Actor secundario: Módulo de validación criptográfica

Precondiciones:

- El archivo `.py` debe ser válido y estar abierto en la IDE.

Postcondiciones:

- El archivo queda actualizado y su hash o firma también se actualiza.

Flujo principal:

1. El estudiante escribe o modifica código en la IDE.
2. El sistema registra que la edición se realiza desde un entorno autorizado.
3. El sistema actualiza el contenido del archivo.
4. El módulo criptográfico recalcula la firma o hash.
5. El sistema guarda el archivo con su nueva validación.

Flujos alternos:

- Si el archivo fue alterado externamente, el sistema impide la edición.
- Si ocurre un error al generar la nueva firma, el sistema no guarda el cambio.

1.9.5. CU-05: Bloquear edición de archivo modificado externamente

Objetivo: Evitar que se utilicen archivos Python alterados fuera del sistema.

Actor principal: Módulo de validación criptográfica

Actor secundario: Estudiante

Precondiciones:

- Existe un archivo .py previamente registrado por el sistema.

Postcondiciones:

- El archivo queda bloqueado para edición si no supera la validación.

Flujo principal:

1. El sistema intenta abrir o validar un archivo .py.
2. El módulo verifica la firma o hash del archivo.
3. El sistema detecta discrepancias con la firma almacenada.
4. El sistema bloquea la edición del archivo.
5. El sistema informa que el archivo fue modificado fuera del entorno autorizado.

Flujos alternos:

- Si la validación es correcta, el archivo se abre normalmente.

1.9.6. CU-06: Ejecutar casos de prueba

Objetivo: Permitir al estudiante verificar si su solución cumple con los criterios definidos.

Actor principal: Estudiante

Actor secundario: Motor de casos de prueba

Precondiciones:

- El archivo debe existir y contener código ejecutable.

Postcondiciones:

- El sistema muestra el resultado de los casos de prueba.

Flujo principal:

1. El estudiante selecciona la opción de ejecutar pruebas.
2. El sistema envía el código al motor de pruebas.
3. El motor ejecuta los casos configurados.
4. El sistema presenta los resultados al estudiante.

Flujos alternos:

- Si el código tiene errores de sintaxis, el sistema informa que no se pudieron ejecutar las pruebas.
- Si no hay casos de prueba configurados, el sistema informa esa condición.

1.9.7. CU-07: Realizar commit de cambios

Objetivo: Permitir al estudiante guardar una versión de su trabajo en Git.

Actor principal: Estudiante

Actor secundario: Repositorio Git

Precondiciones:

- El estudiante ha realizado cambios válidos en sus archivos.

Postcondiciones:

- Se registra un nuevo *commit* en el repositorio asociado.

Flujo principal:

1. El estudiante selecciona la opción de realizar *commit*.
2. El sistema valida que los archivos sean permitidos y estén íntegros.
3. El estudiante ingresa un mensaje de *commit*.
4. El sistema envía los cambios al repositorio Git.

5. El repositorio registra la nueva versión.

6. El sistema confirma el *commit* exitoso.

Flujos alternos:

- Si el archivo no pasa la validación criptográfica, el *commit* es rechazado.
- Si falla la conexión con Git, el sistema informa el error.

1.9.8. CU-08: Gestionar proyectos colaborativos

Objetivo: Permitir que varios estudiantes trabajen conjuntamente en un mismo proyecto.

Actor principal: Estudiante

Actor secundario: Repositorio Git

Precondiciones:

- El profesor ha habilitado un proyecto grupal.

Postcondiciones:

- Los miembros del equipo pueden trabajar sobre el mismo proyecto con control de versiones.

Flujo principal:

1. El estudiante accede a un proyecto colaborativo asignado.
2. El sistema habilita el entorno compartido del proyecto.
3. Los miembros del equipo editan archivos desde la IDE.
4. El sistema coordina cambios mediante Git.
5. Cada miembro puede registrar *commits* según sus permisos.

Flujos alternos:

- Si el estudiante no pertenece al equipo, el sistema niega el acceso.
- Si existe conflicto de versiones, el sistema notifica la situación.

1.9.9. CU-09: Crear asignación

Objetivo: Permitir al profesor publicar un nuevo ejercicio o proyecto.

Actor principal: Profesor

Precondiciones:

- El profesor ha iniciado sesión y administra un curso.

Postcondiciones:

- La asignación queda disponible para los estudiantes del curso.

Flujo principal:

1. El profesor ingresa al curso.
2. Selecciona la opción de crear asignación.
3. Ingresa título, descripción y parámetros del ejercicio.
4. El sistema guarda la asignación.
5. El sistema la publica en el *dashboard* de los estudiantes.

Flujos alternos:

- Si faltan datos obligatorios, el sistema no permite guardarla.

1.9.10. CU-10: Configurar casos de prueba

Objetivo: Permitir al profesor definir pruebas automáticas para una asignación.

Actor principal: Profesor

Actor secundario: Motor de casos de prueba

Precondiciones:

- La asignación debe existir.

Postcondiciones:

- Los casos de prueba quedan asociados a la asignación.

Flujo principal:

1. El profesor abre una asignación existente.
2. Selecciona la opción de configurar casos de prueba.
3. Define entradas, salidas esperadas y condiciones de validación.
4. El sistema almacena la configuración.
5. El motor de pruebas queda preparado para utilizarlos.

Flujos alternos:

- Si la configuración es inválida, el sistema solicita corrección.

1.9.11. CU-11: Agregar comentarios al trabajo del estudiante

Objetivo: Permitir al profesor brindar retroalimentación sobre el código del estudiante.

Actor principal: Profesor

Precondiciones:

- El estudiante debe haber trabajado sobre la asignación.

Postcondiciones:

- Los comentarios quedan visibles para el estudiante.

Flujo principal:

1. El profesor abre la solución de un estudiante.
2. Revisa el código o el historial de cambios.
3. Escribe comentarios de retroalimentación.
4. El sistema guarda los comentarios.
5. El estudiante puede visualizarlos desde la plataforma.

Flujos alternos:

- Si no existe solución disponible, el sistema informa que no hay contenido para comentar.

1.9.12. CU-12: Gestionar llaves de validación del curso

Objetivo: Permitir al profesor administrar las llaves o parámetros criptográficos del curso.

Actor principal: Profesor

Actor secundario: Módulo de validación criptográfica

Precondiciones:

- El profesor administra un curso activo.

Postcondiciones:

- El curso queda asociado a una llave pública o mecanismo de validación.

Flujo principal:

1. El profesor accede a la configuración del curso.
2. Registra o actualiza la llave pública del curso.
3. El sistema almacena la llave.
4. El módulo de validación la utiliza para verificar archivos del curso.

Flujos alternos:

- Si la llave no es válida, el sistema no la registra.

1.10. Requerimientos No Funcionales

Los requerimientos no funcionales se organizan siguiendo el esquema FURPS+ (Funcionalidad, Usabilidad, Confiabilidad, Rendimiento, Soporte y restricciones adicionales). Estos atributos de calidad condicionan el diseño y la implementación de la plataforma.

1.10.1. Funcionalidad (Functionality)

Código de requisito	RNF-F01
Nombre de requisito	Seguridad en comunicaciones
Tipo	No funcional – Funcionalidad
Descripción	Toda comunicación entre la aplicación de escritorio y los servicios remotos (repositorio Git, servidor de asignaciones) deberá realizarse utilizando protocolos cifrados (HTTPS/TLS 1.2 o superior).
Prioridad del requisito	Alta

Código de requisito	RNF-F02
Nombre de requisito	Integridad criptográfica
Tipo	No funcional – Funcionalidad
Descripción	La firma digital de cada archivo .py se calculará usando el algoritmo SHA-256. La clave o mecanismo de firma empleado para el curso será administrado exclusivamente por el profesor y su valor nunca será expuesto en la interfaz del estudiante.
Prioridad del requisito	Alta

Código de requisito	RNF-F03
Nombre de requisito	Aislamiento del entorno de ejecución
Tipo	No funcional – Funcionalidad
Descripción	Los programas Python ejecutados dentro de la terminal interactiva deberán correr en un proceso aislado con permisos restringidos, de manera que no puedan acceder a recursos del sistema operativo fuera del directorio de trabajo del proyecto activo.
Prioridad del requisito	Alta

Código de requisito	RNF-F04
Nombre de requisito	Control de acceso basado en roles
Tipo	No funcional – Funcionalidad
Descripción	El sistema implementará un esquema de control de acceso que garantice que las operaciones disponibles en la interfaz sean exclusivamente las correspondientes al rol del usuario autenticado (Estudiante o Profesor).
Prioridad del requisito	Alta

1.10.2. Usabilidad (Usability)

Código de requisito	RNF-U01
Nombre de requisito	Curva de aprendizaje reducida
Tipo	No funcional – Usabilidad
Descripción	Un estudiante de primer semestre sin experiencia previa en entornos de desarrollo integrado deberá ser capaz de completar su primera asignación dentro de la plataforma en un tiempo máximo de 30 minutos, sin requerir capacitación adicional más allá del tutorial integrado.
Prioridad del requisito	Media

Código de requisito	RNF-U02
Nombre de requisito	Mensajes de error comprensibles
Tipo	No funcional – Usabilidad
Descripción	Todos los mensajes de error y advertencia del sistema se presentarán en español, con una descripción clara de la causa del problema y una sugerencia de acción correctiva cuando aplique.
Prioridad del requisito	Media

Código de requisito	RNF-U03
Nombre de requisito	Consistencia visual
Tipo	No funcional – Usabilidad
Descripción	La interfaz seguirá lineamientos de diseño coherentes (paleta de colores, tipografía y distribución de elementos) en todas las pantallas de la aplicación, de acuerdo con las guías de estilo definidas durante el diseño de la plataforma.
Prioridad del requisito	Baja

Código de requisito	RNF-U04
Nombre de requisito	Accesibilidad básica
Tipo	No funcional – Usabilidad
Descripción	La aplicación deberá soportar ajuste del tamaño de fuente dentro del editor de código y ofrecer contraste visual suficiente para usuarios con sensibilidades de visión de color comunes.
Prioridad del requisito	Media

1.10.3. Confiabilidad (Reliability)

Código de requisito	RNF-R01
Nombre de requisito	Disponibilidad en períodos de evaluación
Tipo	No funcional – Confiabilidad
Descripción	La plataforma deberá mantener una disponibilidad mínima del 99 % durante las semanas de entrega de asignaciones, según el calendario académico del curso.
Prioridad del requisito	Alta

Código de requisito	RNF-R02
Nombre de requisito	Guardado automático
Tipo	No funcional – Confiabilidad
Descripción	La IDE realizará un guardado automático local del archivo en edición cada 60 segundos, evitando la pérdida de progreso ante cierres inesperados de la aplicación.
Prioridad del requisito	Alta

Código de requisito	RNF-R03
Nombre de requisito	Tolerancia a fallos de conectividad Git
Tipo	No funcional – Confiabilidad
Descripción	En caso de que la operación de <i>commit</i> o <i>pull</i> falle por problemas de red, el sistema deberá conservar los cambios localmente y notificar al usuario para que reintente la operación cuando la conexión sea restaurada.
Prioridad del requisito	Media

Código de requisito	RNF-R04
Nombre de requisito	Preservación ante corrupción
Tipo	No funcional – Confiabilidad
Descripción	Si el módulo de validación detecta corrupción o alteración de una firma digital, el archivo original firmado almacenado en el servidor permanecerá inalterable, garantizando la trazabilidad del trabajo del estudiante.
Prioridad del requisito	Alta

1.10.4. Rendimiento (Performance)

Código de requisito	RNF-P01
Nombre de requisito	Respuesta del editor
Tipo	No funcional – Rendimiento
Descripción	Las operaciones de escritura y edición dentro de la IDE deberán reflejarse en pantalla en un tiempo máximo de 200 ms, incluyendo el resaltado de sintaxis, para garantizar una experiencia de edición fluida.
Prioridad del requisito	Alta

Código de requisito	RNF-P02
Nombre de requisito	Ejecución de casos de prueba
Tipo	No funcional – Rendimiento
Descripción	El motor de casos de prueba deberá completar la ejecución de un conjunto de hasta diez pruebas sobre un programa Python de complejidad introductoria en no más de 10 segundos.
Prioridad del requisito	Media

Código de requisito	RNF-P03
Nombre de requisito	Carga del dashboard
Tipo	No funcional – Rendimiento
Descripción	El <i>dashboard</i> de cursos y asignaciones deberá cargarse completamente en un tiempo máximo de 3 segundos bajo condiciones normales de red.
Prioridad del requisito	Media

Código de requisito	RNF-P04
Nombre de requisito	Validación criptográfica eficiente
Tipo	No funcional – Rendimiento
Descripción	El cálculo y la verificación de la firma SHA-256 de un archivo <code>.py</code> de hasta 500 KB no deberá agregar una latencia perceptible (mayor a 100 ms) al proceso de guardado del archivo.
Prioridad del requisito	Alta

1.10.5. Soporte (Supportability)

Código de requisito	RNF-S01
Nombre de requisito	Compatibilidad de plataformas
Tipo	No funcional – Soporte
Descripción	La aplicación de escritorio desarrollada en C# deberá ejecutarse correctamente en sistemas operativos Windows 10 (64 bits) o superior, y en macOS 12 (Monterey) o superior, mediante el <i>runtime</i> multiplataforma .NET 8.
Prioridad del requisito	Alta

Código de requisito	RNF-S02
Nombre de requisito	Modularidad del código fuente
Tipo	No funcional – Soporte
Descripción	La arquitectura del sistema seguirá un diseño en capas claramente separadas (presentación, lógica de negocio y acceso a datos/servicios), de manera que los módulos puedan ser actualizados o sustituidos de forma independiente.
Prioridad del requisito	Media

Código de requisito	RNF-S03
Nombre de requisito	Documentación técnica
Tipo	No funcional – Soporte
Descripción	Todos los módulos principales de la plataforma contarán con documentación <i>in-line</i> completa (XML Doc Comments en C#) y se generará documentación técnica navegable al menos para las interfaces públicas de cada componente.
Prioridad del requisito	Baja

Código de requisito	RNF-S04
Nombre de requisito	Actualizaciones de la aplicación
Tipo	No funcional – Soporte
Descripción	La plataforma incorporará un mecanismo de verificación de actualizaciones al inicio, que notifique al usuario cuando exista una nueva versión disponible y facilite su instalación sin pérdida de datos locales.
Prioridad del requisito	Baja

1.10.6. Restricciones adicionales (+)

Código de requisito	RNF-C01
Nombre de requisito	Lenguaje e implementación
Tipo	No funcional – Restricción
Descripción	La aplicación deberá desarrollarse en C# utilizando .NET 8 como plataforma de ejecución. La interfaz gráfica de escritorio se construirá con WPF (Windows Presentation Foundation) o con MAUI para soporte multiplataforma.
Prioridad del requisito	Alta

Código de requisito	RNF-C02
Nombre de requisito	Requisitos de Python
Tipo	No funcional – Restricción
Descripción	El entorno cliente deberá tener instalado Python 3.10 o superior. La plataforma verificará la presencia del intérprete al iniciar y notificará al usuario si no está disponible.
Prioridad del requisito	Alta

Código de requisito	RNF-C03
Nombre de requisito	Integración con Git
Tipo	No funcional – Restricción
Descripción	El sistema utilizará la instalación local de Git (<code>git</code> en el PATH del sistema) para ejecutar operaciones de control de versiones. Se soportarán repositorios en GitHub y GitLab.
Prioridad del requisito	Alta

Código de requisito	RNF-C04
Nombre de requisito	Requisitos mínimos de hardware
Tipo	No funcional – Restricción
Descripción	La plataforma deberá funcionar correctamente en equipos con al menos 4 GB de RAM, procesador de 1.6 GHz de doble núcleo y 500 MB de espacio libre en disco, con el fin de ser accesible en los laboratorios de computación de la institución.
Prioridad del requisito	Media

Código de requisito	RNF-C05
Nombre de requisito	Restricción de archivos
Tipo	No funcional – Restricción
Descripción	El sistema únicamente procesará, abrirá y validará archivos con extensión <code>.py</code> . No se admitirá la importación de archivos de código en otros lenguajes ni el uso de paquetes no instalados previamente en el entorno del curso.
Prioridad del requisito	Alta

1.11. Diagrama de Casos de Uso

El diagrama de casos de uso presenta las principales interacciones entre los actores identificados y las funcionalidades del sistema propuesto. En él se representan las acciones que pueden realizar tanto los estudiantes como los profesores dentro de la plataforma, así como la relación con sistemas externos como el repositorio Git y el motor de casos de prueba. Además, se incluyen relaciones de tipo «**include**» y «**extend**» para reflejar dependencias y extensiones entre algunos casos de uso.

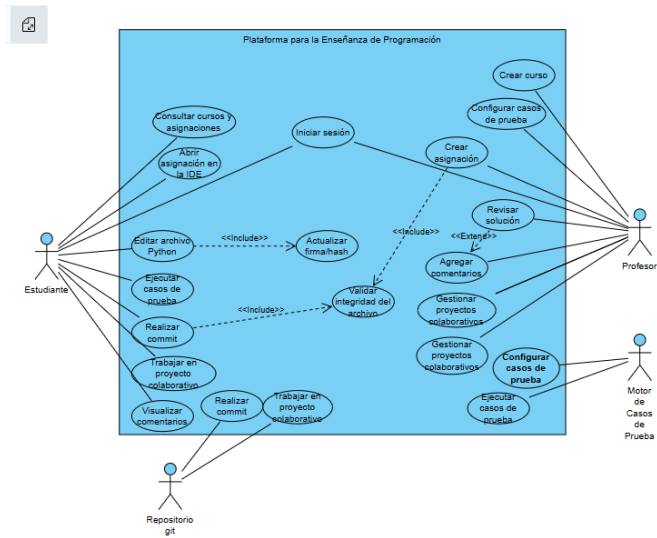


Figura 1.1: Diagrama de casos de uso del sistema propuesto

Referencias

Bruegge, B., y Dutoit, A. H. *Object-Oriented Software Engineering Using UML, Patterns, and Java*. 3ra edición, Pearson, 2010. Object Management Group. *Unified Modeling Language (UML) Version 2.5.1*. 2017. Disponible en: <https://www.omg.org/spec/UML/2.5.1>. Consultado en marzo de 2026. ISO/IEC. *ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. 2011. Disponible en: <https://www.iso.org/standard/35733.html>. Consultado en marzo de 2026. OpenAI. *ChatGPT*. 2026. Disponible en: <https://openai.com/chatgpt/>. Consultado en marzo de 2026. Object Management Group. *About the Unified Modeling Language Specification, Version 2.5.1*. 2017. Disponible en: <https://www.omg.org/spec/UML/2.5.1/About-UML>. Consultado en marzo de 2026.